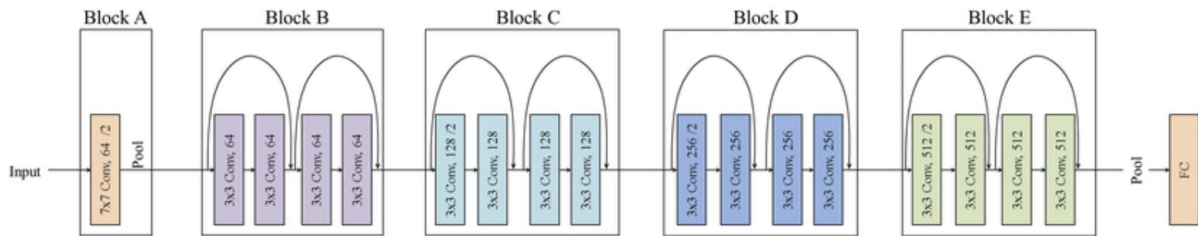


HW6: Implementing ResNet18 with Triton

1. 목표: Triton을 이용하여 ResNet18 구조를 병렬화하여 가속하는 Program을 작성한다.

2. ResNet18 구조



ResNet18 구조

MGP강의는 AI 모델 강의가 아니므로, 위 아키텍처에 대한 상세한 분석은 진행하지 않고, 위 아키텍처를 이용 시 어느 연산의 속도를 최적화할 때 가장 효율이 좋은지를 분석한다.

2.1. 연산의 종류

Block A: 64개 $7 \times 7 \times 3$ 2D Conv -> Batch Norm -> 3×3 Max Pooling

Block B: 64개 $3 \times 3 \times 64$ 2D Conv -> Batch Norm -> ReLU : 4번

Block C: 128개 $3 \times 3 \times 64$ 2D Conv -> Batch Norm -> ReLU : 1번

128개 $3 \times 3 \times 128$ 2D Conv -> Batch Norm -> ReLU : 3번

128개 $1 \times 1 \times 64$ 2D Conv

Block D: 256개 $3 \times 3 \times 128$ 2D Conv -> Batch Norm -> ReLU : 1번

256개 $3 \times 3 \times 256$ 2D Conv -> Batch Norm -> ReLU : 3번

256개 $1 \times 1 \times 128$ 2D Conv

Block E: 512개 $3 \times 3 \times 256$ 2D Conv -> Batch Norm -> ReLU : 1번

512개 $3 \times 3 \times 512$ 2D Conv -> Batch Norm -> ReLU : 3번

512개 $1 \times 1 \times 256$ 2D Conv

FC layer: $7 \times 7 \times 512$ Feature Map과 1000개 뉴런과 Fully Connected MAC 연산

즉, 연산의 경우 2D Convolution에서 가장 많은 연산량이 발생하므로, Conv2d를 최적화 시키는 것이 성능에 가장 크게 영향을 줄 것임을 알 수 있다.

3. Triton의 Hardware level 코드 구현 방식

[번외 과제인 Triton Puzzle의 예제]

```
1. def sum_spec(x: Float32[Tensor, "4 200"]) -> Float32[Tensor, "4"]:  
2.     return x.sum(1)  
3.  
4. @triton.jit  
5. def sum_kernel(x_ptr, z_ptr, N0, N1, T, B0: tl.constexpr, B1: tl.constexpr):  
6.     pid = tl.program_id(0) # 각 블록이 하나의 row를 담당  
7.     row = pid * B0 + tl.arange(0, B0)  
8.     mask_row = row < N0  
9.  
10.    acc = tl.zeros([B0], dtype=tl.float32) # 누적합 저장용  
11.  
12.    for t in range(0, T, B1): # B1 단위로 열을 순회  
13.        offs_t = tl.arange(0, B1)  
14.        col = t + offs_t  
15.        mask_col = col < T  
16.  
17.        idx = row[:, None] * T + col[None, :] # 2D index → flat  
18.        mask = mask_row[:, None] & mask_col[None, :]  
19.  
20.        x = tl.load(x_ptr + idx, mask=mask, other=0.0) # 유효한 위치만 더함  
21.        acc += tl.sum(x, axis=1) # 열 방향으로 합산  
22.  
23.        tl.store(z_ptr + row, acc, mask=mask_row) # 최종 합 저장  
24.  
25.    return  
26.  
27. test(sum_kernel, sum_spec, B={"B0": 1, "B1": 32}, nelem={"N0": 4, "N1": 32, "T": 200})
```

먼저, triton은 `import triton.language as tl`을 통해 연산, indexing, masking, memory access등을 수행할 수 있다. 아래는 자주 쓰이는 기능들을 정리한 표이다.

Category	Examples	Description
Thread/Grid Management	<code>tl.program_id(0)</code>	ID of the current program block (functions like a thread block)
Memory Access	<code>tl.load</code> , <code>tl.store</code>	Load from and store to global memory
Vector/Matrix Ops	<code>tl.sum</code> , <code>tl.dot</code> , <code>tl.zeros</code>	Arithmetic operations such as sum, dot product, and initialization
Logic/Condition	<code>tl.where</code> , <code>tl.maximum</code>	Conditional operations like ReLU
Vector Generation	<code>tl.arange</code> , <code>tl.full</code>	Generate index vectors or constant tensors

위는 크기가 (4, 200)인 2차원 Tensor의 row별 합계를 계산하는 Triton Kernel 예제이다.

3.1) Parameter는 다음과 같이 선언된다.

x_ptr: 입력 Tensor의 Pointer

z_ptr: 출력 Tensor의 Pointer

N0: row의 개수

T: 총 column의 수

B0: 1개의 Block이 처리할 row 수

B1: 1개의 loop에서 처리할 column 수

즉, CUDA Programming과 비슷하게 Block 단위로 큰 작업 중 일부를 수행하는 것을 알 수 있다.

3.2) index 계산

program_id(0)은 1D grid 상에서 현재 Block의 ID를 나타내는 것으로 이를 통해 element의 index를 계산($row = pid * B0 + tl.arange(0, B0)$)할 수 있다. mask_row의 경우에는 row가 N0를 넘어가면 out of bounds 에러가 발생하기에 사용하는 Mask 변수이다.

3.3) 실제 summation계산

for loop는 triton kernel에서 1개의 Block이 실제로 sum연산을 수행하는 부분이다.

CUDA Programming에서는 thread단위로 Kernel이 실행되었지만, triton의 경우에는 CUDA의 thread Block과 유사한 단위(= program instance)로 Kernel이 실행된다고 한다.

따라서, $B0 \times T$ 개의 elements를 1개의 program instance가 처리하고, 1번의 iteration당 B1개의 element를 처리하면 loop를 돈다. 1번의 iteration당 $B0 \times B1$ 개의 elements로 나뉘어서 처리하는 이유는 GPU의 shared memory나 warp를 고려하기 때문이다.

Triton은 익숙하지 않은 언어이기에 이 예제 코드에 관해서만큼은 좀 더 detail하게 살펴본다.

3.3.1) $tl.arange(0, B1)$ 의 경우, 현재 tile에서 처리할 column index를 생성한다.

예를 들어 $B1 = 32, t = 64$ 일 때, $tl.arange(0, 32)$ 는 $[0, 1, \dots, 31]$ 의 결과를 생성하고,

$col = 64 + [0, 1, \dots, 31]$ 는 $[64, 65, \dots, 95]$ 가 된다.

3.3.2) $mask_col = col < T$ 는 boundary check를 위한 masking 연산으로, col vector의 각 element가 유효한 범위 안에 있는지를 확인하여, out of bounds memory access를 방지한다.

3.3.3) $idx = row[:, None] * T + col[None, :]$ 는 1D Memory Index로 변환하는 역할을 하고, $row[:, None]$ 의 shape은 $[B0, 1]$ 이고, $col[None, :]$ 의 shape은 $[1, B1]$ 이므로 shape $[B0, B1]$ 인 idx정보를 가진 Tensor가 형성된다.

3.3.4) $mask = mask_row[:, None] \& mask_col[None, :]$ 는 현재 iteration에서 처리하는 $B0 \times B1$ 의 element들이 유효한 위치에 있는지 판단하는 mask 정보를 지닌 shape $[B0, B1]$ Tensor를 형성한다.

3.3.5) $x = tl.load(x_ptr + idx, mask=mask, other=0.0)$ 는 x의 시작 주소인 x_ptr로부터 idx에 따라, element들을 global memory에서 load한다. 더불어, 앞서 살펴본 mask의 값(True, False)에 따라서, True이면 load를 수행하고, False이면 그냥 $other = 0$ 을 채워 넣는다.

여기서, CUDA와의 차이점이 존재한다. CUDA의 경우에는 load나 store의 경우 `__shared__`,

`__global__` 와 같은 code를 작성하여, memory를 직접 지정해줄 수 있었지만, triton은 그러한 기능은 제한적이다. Triton의 경우, data 접근 pattern이나, data 크기에 따라 내부적으로 shared memory, register 사용 여부가 결정된다고 한다.

3.3.6) `acc += tl.sum(x, axis=1)`은 각 row에 대해 row-wise (= B1개 elements)의 sum를 계산한다. 결과는 shape [B0]인 1차원 vector가 나오게 된다.

3.3.7) `tl.store(z_ptr + row, acc, mask=mask_row)`는 acc의 결과를 `z_ptr`를 기준으로 row의 offset을 두고 store가 된다. row의 shape은 [B0]이기에, 이 code를 통해서 B0의 element들이 각각 알맞은 주소에 store된다. 더불어, `mask=mask_row`를 통해 유효한 위치의 data만 store이 진행된다.

마지막으로, `@triton.jit`는 그 안의 code를 Triton Compiler가 분석하여 GPU에서 병렬로 실행될 수 있는 kernel로 변환할 수 있게 하는 문구이다.

4. How did you implemented your parallel algorithm

4.1. AvgPool2d.py

AvgPool의 경우에는 input `x` (shape: [N, C, H, W])에 대해, kernel size (`kH`, `kW`)와 stride (`sH`, `sW`)를 고려하여 2D average pooling을 계산해야 한다.

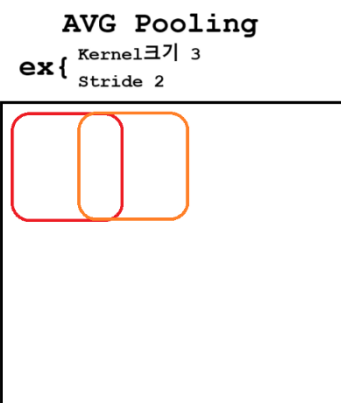
Kernel의 동작을 정의할 때, CUDA Programming에서와 같이 Output Feature Map의 element를 기준으로 Indexing을 계산하여 연산을 진행한다. 하지만, Triton에서는 Thread 단위가 아니라 Block단위와 유사하게 Kernel이 실행되므로, Output Feature Map에서 element 구간을 정의해야 한다.

현재 본인의 코드는 병렬화는 row를 기준으로 구성하였고, 하나의 Block이 Output Feature Map의 Column들의 특정구간을 연산하도록 하였다. 그림으로 표현하면 다음과 같다.

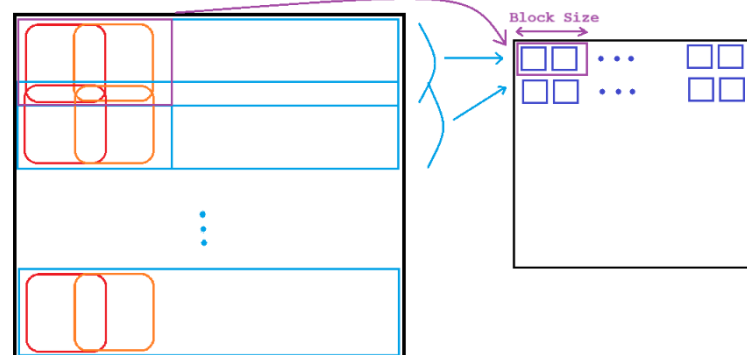
오른쪽 그림에서, `_avgpool2d_kernel`이 각각 Output의 1개 row에 대한, Block Size만큼의 elements를 계산하는 것을 표현하였다. 만약 Block Size = 256개이면 1개의 `_avgpool2d_kernel`은 $256 \times kH \times kW$ 만큼의 elements를 input feature map에서 load를 하고 avg연산 후 output element 256개를 store을 할 것이다.

참고로, `kH`, `kW`는 Filter의 width, height이다.

따라서, Block Size에 따라 load하는 element의 개수가 달라지기에, Block Size를 잘 설정하여, Triton이 내부적으로 GPU의 shared memory 또는 register를 잘 활용하도록 해야 한다.



Stride < Kernel Size 가정하고 그림



또한, 전체 program 수는 아래와 같이 계산되어, Block Size와 반비례하여 생성된다. 이는 GPU에서 몇 개의 thread가 병렬적으로 동작하는지와 비슷한 개념으로, Block Size가 너무 작으면, 너무 많은 Program이 생성되어, Program들이 한 번에 병렬적으로 실행되지 않고, 부분적으로만 실행될 수 있다. 따라서, 이를 고려하여 한 번에 많은 Program들이 병렬로 실행될 수 있도록 해야 한다.

```
WB = (W_out + BLOCK_SIZE - 1) // BLOCK_SIZE
num_programs = N * C * H_out * WB
```

4.2. BatchNorm2d.py

BatchNorm2d 연산은 input X에 대해, running mean, running variance, scale, bias 값 등을 기반으로 normalization을 수행한다.

이는 Batch Norm의 경우 channel 별로 normalization된다는 특징이 있다. 따라서, channel의 index를 정확히 계산해야 한다.

본인은 아래와 같이 channel의 index를 계산하였다.

```
pid = tl.program_id(0)
# Total elements per channel
T = H * W
NB = tl.cdiv(T, BLOCK_SIZE)
nc_index = pid // NB
tile_index = pid % NB
# Recover batch index n and channel index c
n = nc_index // C
c = nc_index % C
```

T는 1개의 (n, c)가 가지는 image 1장의 크기로, NB는 이 이미지 1장을 처리하는데 필요한 Block 개수를 의미한다. 따라서, pid는 Block의 개수를 의미하므로, pid / NB를 하게 되면, n x c의 index를 구할 수 있다. 이 n x c의 index를 C로 나눈 나머지가 channel의 index가 된다.

이제 각 Block이 어떻게 연산하는지를 살펴본다.

연산의 경우 오른쪽 그림에서처럼 BLOCK_SIZE 길이만큼의 input의 elements를 가져와서 처리한다.

예를 들어 BLOCK_SIZE = 16이라면,

하나의 kernel은 (n, c)에 해당하는 input feature map에서 H×W 중 16개의 elements를 load하고, BatchNorm 연산을 수행한 뒤, 같은 offset을 이용해 y 배열에 결과값 16개를 store한다.

```
if, H = 8, W = 8
Block Size = 16
[각 Block 이 2개의 row를 연산]
```

row 0	Block 0
row 1	
row 2	Block 1
row 3	
row 4	Block 2
row 5	
row 6	Block 3
row 7	

참고로, 각각의 Block은 2D가 아니라 1D로 설정하여 row-wise하게 elements를 load한다.

앞서 살펴보았던 AvgPool2d와 마찬가지로 BLOCK_SIZE가 클수록 한 번에 처리하는 input 영역이 넓어지고, program 수는 적어진다. 반면 BLOCK_SIZE가 작으면 program 수가 많아지며, GPU에서

동시에 처리할 수 있는 program 수를 초과할 경우 병렬성이 떨어지고 성능이 감소할 수 있다.

4.3. Conv2d.py

Conv2d는 이번 실습에서 가장 성능에 큰 영향을 주는 부분이다.

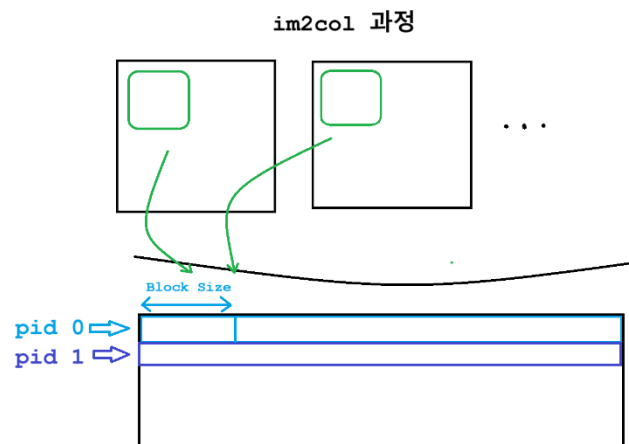
ResNet의 경우 channel이 많고 input Batch의 수 또한 많기에, Direct Convolution이 아니라 im2col + gemm 방식을 사용하는 것이 더 효과적일 것이라고 판단하였다. 따라서, 전체적인 Conv2d는 "im2col 변환 kernel -> weight shape 변환 kernel -> gemm 연산 kernel -> 연산 결과를 reshape하는 kernel"과 같이 구성하였다.

4.3.1 im2col kernel

현재 im2col kernel은 im2col에 transpose를 취한 형태로 결과가 나오도록 작성되어 있다. 이는 row-wise로 tl.load와 tl.store를 해야 연속적인 memory 접근을 하여 더 빨라질 것 같다고 생각하였기 때문이다.

im2col kernel에서 각 program은 1개의 output row를 담당하게 하였다. 따라서, 총 pid 수는 $N * H_{out} * W_{out}$ 로 설정했다.

filter 위치에 따른 index는 오른쪽과 같이 작성하였고, 여기서 봐야 하는 것은 tl.load 및 tl.store에서 모두 k_idx에 따라 coalescing을 지키며 load & store가 일어나고 있어, 효율적인 memory 접근을 하고 있다는 것이다.



```

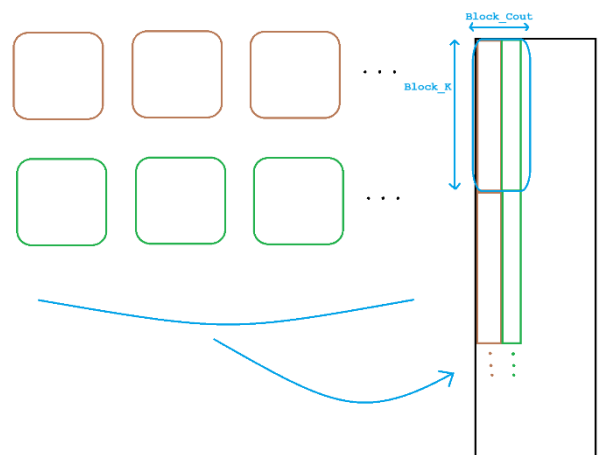
k_idx = i + offs          # column index of current row
ci = k_idx // (kh * kw)   # input channel index
rem = k_idx % (kh * kw)   # position of the filter within the channel, (= which Filter)
kh = rem // kw            # position within the filter height
kw = rem % kw             # position within the filter width
hi = ho * sh - pH + kh * dH # input height index
wi = wo * sw - pw + kw * dW # input width index
offset = ((n * C + ci) * H + hi) * W + wi # memory offset for x[n, ci, hi, wi]
x_vals = tl.load(x_ptr + offset, mask=is_valid, other=0.0)
offset_col = row * K + k_idx
tl.store(col_ptr + offset_col, x_vals, mask=k_idx < K)

```

4.3.2 weight transpose kernel

현재 weight transpose kernel은 im2col + gemm을 이용한 Conv2D 연산 시 사용되는 filter weight의 shape $[C_{out}, C_{in}, kH, kW]$ 을 matmul에 적합한 형태인 $[K, C_{out}]$ 으로 변환하기 위해 사용되는 kernel이다.

여기서는 program이 output shape인 $[K, C_{out}]$ 을 block_size로 나눈 $\left(\frac{K}{Block_K}\right) \times \left(\frac{C_{out}}{Block_C}\right)$ 만큼 실행되고, 1개의 program은 $Block_K \times Block_C$ 만큼의 element를 재 배열하여 store한다.



weight transpose의 경우 index는 오른쪽 그림과 같이 계산하였고, 이 경우 load에서는 co를 기준으로 load가 되고 있기에, coalescing이 지켜지지 않아 비효율적인 memory 접근이 될 수밖에 없다. 반면, store의 경우에는 coalescing이 지켜져 효율적이다. 만약, load의 memory 접근을 coalescing하도록 바꾼다면, 성능향상을 기대해볼 수 있을 것이다.

```

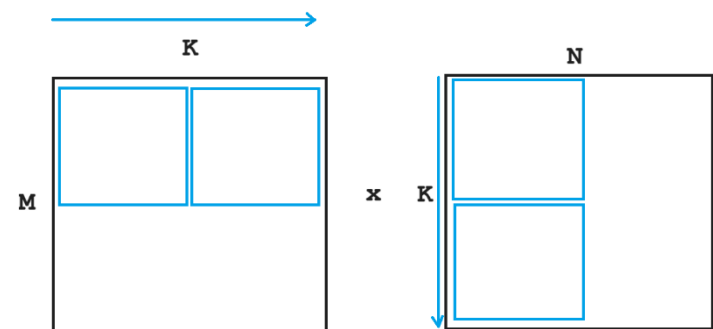
offs_k = pid_k * BLOCK_K + tl.arange(0, BLOCK_K) # (K,)
offs_c = pid_c * BLOCK_C + tl.arange(0, BLOCK_C) # (C_out,)
kk = offs_k[:, None] # (BLOCK_K, 1)
co = offs_c[None, :] # (1, BLOCK_C)
# Compute ci, kh, kw for each K
ci = kk // (KH * KW)
rem = kk % (KH * KW)
kh = rem // KW
kw = rem % KW
# Compute the input weight offset
weight_offset = ((co * C_in + ci) * KH + kh) * KW + kw # shape (BLOCK_K, BLOCK_C)
vals = tl.load(weight_ptr + weight_offset, mask=mask, other=0.0)
# Store at transposed location: (K, C_out)
trans_offset = kk * C_out + co
tl.store(trans_ptr + trans_offset, vals, mask=mask)

```

하지만, 이번 실습에서는 load의 coalescing까지는 구현하지 못하였다.

4.3.3 gemm kernel

gemm연산의 경우에는 이전 주차 실습에서 적용하였던 tiling을 활용한 blocked matrix multiplication 방식으로 구성하였다. 따라서, matmul kernel의 경우에도 이전의 weight transpose kernel과 유사하게 2D tiling을 하였고, 이를 위해 triton의 pid 또, pid(0)와 pid(1)으로 분리하여 index를 계산하였다.



이와 같이 Block 단위로 load를 하고 store을 하게 되면 Triton에서 compile시 자체적으로 shared memory에 할당할 가능성이 높다고 한다. 따라서, 우리가 GPU programming에서 많이 다루었던, shared memory를 활용한 GEMM연산이 될 가능성이 높아 성능이 좋을 것이라고 예상하여 Blocked Matrix로 코드를 작성하였다.

```

a_ptrs = a_ptr + offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak
b_ptrs = b_ptr + offs_k[:, None] * stride_bk + offs_n[None, :] * stride_bn
c_ptrs = c_ptr + offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn
a = tl.load(a_ptrs, mask=(offs_m[:, None] < M) & ((offs_k[None, :] + k) < K), other=0.0)
b = tl.load(b_ptrs, mask=((offs_k[:, None] + k) < K) & (offs_n[None, :] < N), other=0.0)
tl.store(c_ptrs, acc, mask=(offs_m[:, None] < M) & (offs_n[None, :] < N))

```

Coalescing의 경우 load와 store을 어떻게 하는지 봐야 한다. 오른쪽 code를 보게 되면, load의 경우

offs_k에는 stride만 곱해져 있어 memory 접근 시 연속적인 접근을 할 것임을 알 수 있고, store에서도 offs_n에 stride만 곱해져 있어 memory 접근 시 연속적인 접근을 할 것임을 알 수 있다. 따라서, load & store에서 모두 coalescing이 지켜지고 있다는 것을 확인할 수 있다.

4.4.4 reshape kernel

Im2col + gemm을 하게 되면 연산 결과가 $[M, C_{out}] = [N * H_{out} * W_{out}, C_{out}]$ 형태로 나온다. 따라서 이를 다시 $[N, C_{out}, H_{out}, W_{out}]$ 형태로 재배열하기 위해 reshape kernel을 작성하였다.

이 kernel의 경우에는 1개의 Block이 output의 row-wise order로 Block Size만큼의 element를 처리하도록 하였다. 따라서, 오른쪽 code와 같이 Block Size에 의거하여 offset을 결정하였고, 이 offset에

```

pid = tl.program_id(0)
offs = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE) # (BLOCK_SIZE,)
total = N * C_out * H_out * W_out # total number of output elements

```

따라, input의 batch index, channel index, height index, width index를 계산하여 load를 하였고, Block index에 따라 output에 store을 하였다. 이 경우에는 input_offset에서 w가 가장 빠르게 증가할 텐데, 이 w를 포함하는 m에 C_out이 곱해져 있어, load시에는 coalescing이 깨질 것으로 예상할 수 있다. 반면 store는 우리가 처음에 정의해주었던, Block의 index순서대로 store가 진행되기에 coalescing이 지켜질 것이다.

```
m = n * H_out * W_out + h * W_out + w # compute the flattened index for (n, c, h, w)
input_offset = m * C_out + c # compute the input offset for (n, c, h, w)
output_offset = offs # compute the output offset for the reshaped output
val = tl.load(input_ptr + input_offset, mask=mask, other=0.0)
tl.store(output_ptr + output_offset, val, mask=mask)
```

이와 같이 im2col + gemm구현에서 2번의 reshape가 일어나는데, 여기서 한 번씩 coalescing이 지켜지지 않는다. 따라서, 이 2번의 coalescing을 개선하면 성능이 향상될 것으로 예상된다.

4.4. Linear.py

Linear.py의 경우 먼저, skeleton code로 제공된 오른쪽 code를 보게 되면, float 16으로 x가 들어온다는 것을 확인하고 code를 작성하였다. 따라서, 아래와 같이 compute시에는 32bit로 accumulate할 array를 만들었지만, store시에는 16bit로 타입을 변환하여 store하였다.

```
class TritonLinear(nn.Linear):
    def __init__(
        self,
        in_features: int,
        out_features: int,
        bias: bool = True,
        act_func = None,
        device = 'cuda',
        dtype: torch.dtype = torch.float16,
    ) -> None:
        super().__init__(in_features, out_features, bias, device, dtype)
        self.act_func = act_func

    def forward(self, x):
        return triton_linear(x.to(torch.float16), self.weight.T.contiguous(), self.bias)
```

```
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
tl.store(out_ptrs, tl.cast(acc, tl.float16), mask=mask_out)
```

사실상 Linear.py는 matrix multiplication과 동일하기에 그 외에는 앞서 설명한 mat_mul kernel과 동일하게 구성하였다.

하지만, Linear연산에서는 Block의 Size를 Conv2d에서와 같이 키우면, 오히려 speed가 느려져, Block의 size를 작게 하였다. 이는 아마도 output class가 많지 않기에, Block Size가 클 경우 redundant한 연산이 많이 생기고, 이 결과들은 결국 store되지 않고 버려지기 때문이라고 예상된다.

4.5. MaxPool2d.py

MaxPool2d.py의 경우 앞서 설명한 AvgPool과 거의 유사하게 작성하였다. 차이점은 Max값을 찾아야 하기에 초기화를 -inf로 한 것과 비교 연산을 위해 tl.where()를 이용한 것, 그리고, parameter로 padding여부를 받기에, 오른쪽 코드에서와 같이 kernel실행 전에 padding처리를 한 것이다.

```
H_out = (H + 2 * pH - kH) // sH + 1
W_out = (W + 2 * pW - kW) // sW + 1

out = empty((N, C, H_out, W_out), device=x.device, dtype=x.dtype)
```


4.6. ReLU.py

위 ReLU코드는 skeleton으로 제공되었다. 이는 Block_Size = 1024로 설정하여 하나의 kernel이 1024 elements에 대한 ReLU연산(= tl.where을 이용한다.)을 진행한다. load 및 store는 offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)를 index로 사용해 진행한다. 이 경우 Block Size에 따라 순차적으로 row-wise하게 load를 하고 store을 하므로, coalescing이 지켜졌다.

5. Evaluations of Latency

처음에는 im2col + gemm방식이 아니라 direct convolution 방식을 사용하여, ResNet18 구조를 구현하였다. 더불어, CUDA와 같이 thread단위로 kernel이 구성되는 줄 알아, Block을 이용하지 않고, 각 Kerenl이 오직 output의 1개의 element를 계산하도록 code를 작성하였었다.

이 경우, Kerenl이 오직 output의 1개의 element를 계산하도록 하면, 무한 loop에 걸린 것 마냥 연산 속도가 너무 느려 연산 완료까지 기다릴 수 없었고, 따라서, Program을 Kill해야 했다.

추가로, Block Size를 키워서 구성하여도, Direct Convolution을 사용하게 되면, 2000~3000ms정도의 latency가 발생하여 과제의 조건인 500ms 이하로 달성할 수 없었다.

따라서, im2col + gemm방식을 사용하여 과제의 조건을 달성 하였다. Block Size의 경우, 먼저, 연산에 직접적으로 관련 있는 Conv2d.py와 Linear.py를 제외한 함수에서는 256으로 통일하였다. 그 이유는, CUDA programming과제를 하며, thread Block당 thread의 개수가 128, 256, 512 일 때, 최적의 성능(100% occupancy)를 보였고, Triton의 Kernel은 CUDA에서의 Thread Block과 유사하게 작동하므로, Triton의 Kernel에서 또한 이에 착안하여 Block Size를 256으로 통일한 것이다.

Conv2d.py와 Linear.py의 경우에는 직접적으로 연산을 수행하는 부분이다. 따라서, Block Size는 실험을 통해서 결정을 하였다.

Conv2d.py와 Linear.py에서 Block Size는 다음과 같다.

Conv2d.py		
KERNEL	Block Size Params	Block SIZE
_im2col_kernel	BLOCK_K	64
_transpose_kernel	BLOCK_K, BLOCK_C	64, 64
_matmul_kernel	BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K	64, 64, 32
reshape_output_kernel	BLOCK_SIZE	128
Linear.py		
_linear_tiled_kernel	BLOCK_M, BLOCK_N, BLOCK_K	16, 16, 16

이와 같이 설정하였을 때, Latency는 아래와 같이 측정된다.

왼쪽은 pytorch의 결과, 오른쪽은 triton의 결과

```
$ make clean
find . -type d -name '__pycache__' -exec rm -rf {} +
rm result/accuracy.txt
$ make torch; make torch; make torch; make torch; make torch
python3 run_Torch_ResNet18.py
Loading pretrained model...
Inference Time: 29.1375 ms
Accuracy on test data: 79.73%
Correct.
python3 run_Torch_ResNet18.py
Loading pretrained model...
Inference Time: 28.9392 ms
Accuracy on test data: 79.73%
Correct.
python3 run_Torch_ResNet18.py
Loading pretrained model...
Inference Time: 28.7779 ms
Accuracy on test data: 79.73%
Correct.
python3 run_Torch_ResNet18.py
Loading pretrained model...
Inference Time: 28.9588 ms
Accuracy on test data: 79.73%
Correct.
python3 run_Torch_ResNet18.py
Loading pretrained model...
Inference Time: 28.6882 ms
Accuracy on test data: 79.73%
Correct.
$ █

$ make clean
find . -type d -name '__pycache__' -exec rm -rf {} +
rm result/accuracy.txt
rm: cannot remove 'result/accuracy.txt': No such file or directory
make: *** [Makefile:5: clean] Error 1
$ make triton; make triton; make triton; make triton; make triton
python3 run_Triton_ResNet18.py
Loading pretrained model...
Inference Time: 133.4801 ms
Accuracy on test data: 79.83%
Correct.
python3 run_Triton_ResNet18.py
Loading pretrained model...
Inference Time: 134.2482 ms
Accuracy on test data: 79.83%
Correct.
python3 run_Triton_ResNet18.py
Loading pretrained model...
Inference Time: 134.6195 ms
Accuracy on test data: 79.83%
Correct.
python3 run_Triton_ResNet18.py
Loading pretrained model...
Inference Time: 134.2080 ms
Accuracy on test data: 79.83%
Correct.
python3 run_Triton_ResNet18.py
Loading pretrained model...
Inference Time: 134.9017 ms
Accuracy on test data: 79.83%
Correct.
$ █
```

6. Reasons why Conv2d does not show the efficient performance compared to torch

Triton을 이번에 처음 사용해보면서 느낀 것은 CUDA programming model에 비해 Hardware관점에서 정밀한 제어가 불가능하다고 느꼈다. 즉, shared memory와 같은 요소를 명시적으로 선언하기 어려웠고, thread단위 제어도 불가능하였다.

따라서, Programming이 전체적으로 좀 더 High Level에서 구성되는 느낌이었다. 이러한 이유로 GPU의 Hardware를 더욱 세밀하게 Control하지 못하기에, CUBLAS와 같은 NVIDIA에서 만든 Library를 이용하여 NVIDIA GPU에 최적화된 연산을 수행한 torch에 비해 성능이 더 안 좋게 나왔다고 판단된다.

7. Discussion

이번 실습에서 모든 data형은 32bit로 설정하여 진행하였다. 만약, 이를 16bit로 설정하였다면, accuracy는 약간 떨어지더라도, 속도의 향상이 있었을 것으로 예상된다.

8. References

- Triton Puzzle

- 11_Triton Introduction_mgp_2025.pdf